

Reservations Module

Implementation report and unit-test evidence

This report documents the completed reservation work: automatic table assignment, reservation time validation and persistence, and the tests that verify the behavior. The scope is intentionally limited to the reservations module.

COMPLETED

1

new time field

3

reservation suites

55

total tests passed

0

table/category changes

Executive summary

A reservation can now include a required time in HH:mm format. The value is validated at the API boundary, stored on the reservation, and returned with the saved reservation. During creation, reservations continue to receive an available table with enough capacity through the existing TablesService logic.

No new endpoints, scheduling features, interval model, or table-status workflow were introduced. The existing reservation conflict rule and the existing tables behavior remain unchanged.

Scope guardrail

All implementation changes are inside src/reservations. The category and tables modules were not changed.

1. Implementation details

The change is intentionally small. The table assignment path already existed after the reservation work was completed; this report documents the final reservation contract and adds the time field without introducing a separate feature area.

Request validation

Field	Rule	Purpose
date	Valid date transformed to Date	Reservation calendar date
time	Required; HH:mm, 00:00-23:59	Time of the reservation
quantity	Integer greater than or equal to 1	Capacity used for table selection

Example request

```
POST /api/v1/reservations
Content-Type: application/json

{
  "name": "Juan Gomez",
  "phone": "3000000000",
  "email": "juan@example.com",
  "date": "2026-09-25T00:00:00.000Z",
  "time": "19:00",
  "quantity": 4
}
```

The client does not send a tableId. The server selects an available table and stores it in the reservation relationship.

Creation flow

1	Validate Global validation checks the date, time, quantity, and customer fields.
2	Check existing reservation The pre-existing date conflict rule is preserved.
3	Select table TablesService.findAll() filters AVAILABLE tables with sufficient capacity.
4	Persist The reservation is saved with status PENDING and the selected table.

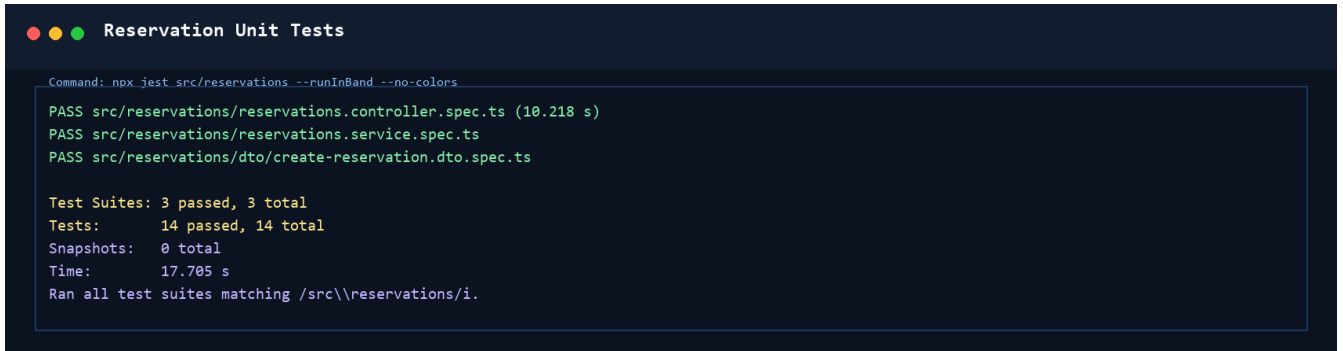
Persistence model

Entity field	Database representation	Notes
date	timestamp	Requested reservation date
time	varchar(5), NOT NULL	Validated HH:mm value
table	FK tableId -> tables.id	Selected server-side; relation is non-null

Entity field	Database representation	Notes
status	enum, default PENDING	Existing reservation lifecycle

2. Unit-test evidence: reservations

The following image is a rendered capture of the actual Jest output from the focused reservation test run. It is included as visual evidence rather than a manually written summary.



```
Reservation Unit Tests

Command: npx jest src/reservations --runInBand --no-colors

PASS src/reservations/reservations.controller.spec.ts (10.218 s)
PASS src/reservations/reservations.service.spec.ts
PASS src/reservations/dto/create-reservation.dto.spec.ts

Test Suites: 3 passed, 3 total
Tests:      14 passed, 14 total
Snapshots:  0 total
Time:       17.705 s
Ran all test suites matching /src\\reservations/i.
```

Figure 1. Real output from `npx jest src/reservations --runInBand --no-colors` captured on 25 September 2026.

Suite	Coverage
reservations.service.spec.ts	Table assignment, no-table conflict, and existing-date conflict
reservations.controller.spec.ts	Controller dependency and instantiation
create-reservation.dto.spec.ts	Positive quantity, invalid time formats, and name validation

Focused run result: 3 suites passed, 14 tests passed.

3. Full-suite verification

The complete test suite was also run after the reservation changes. The image below is the captured output from the real full-suite execution.

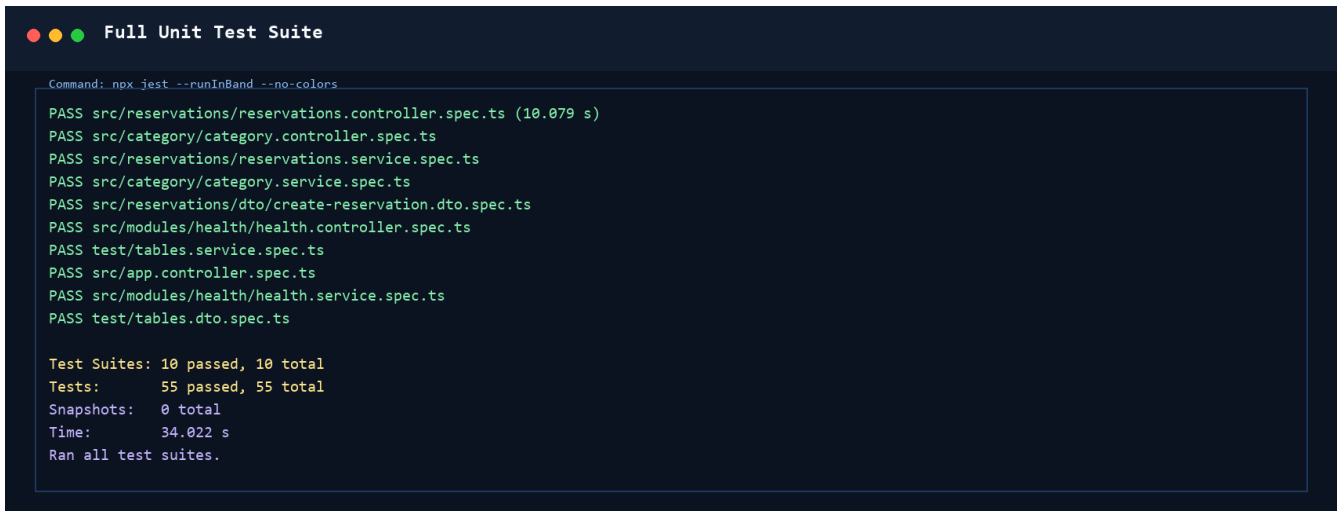


Figure 2. Real output from `npx jest --runInBand --no-colors` captured on 25 September 2026.

Check	Result	Evidence
Focused reservation tests	PASS	3 suites / 14 tests
Complete test suite	PASS	10 suites / 55 tests
TypeScript build	PASS	npm run build
Lint	PASS	0 warnings / 0 errors

Deliberate boundaries

- No new reservation endpoints or features were added.
- No interval fields such as `endAt` or `duration` were introduced.
- No transaction or concurrency model was added; that would require a separate availability policy.
- No changes were made to category or tables logic.
- The existing date conflict rule remains unchanged.

Reproduction commands

```
npm run build
npm run lint
$env:NODE_OPTIONS='--experimental-vm-modules'
npx jest src/reservations --runInBand --no-colors
npx jest --runInBand --no-colors
```

Result: the reservations module is documented, buildable, lint-clean, and covered by passing unit tests, including the new reservation time validation.